

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄奕皓 学号 2024K8009929012 专业 计算机科学与技术
实验项目编号 3 实验名称 定制功能型处理器设计——真实内存、外设与性能计数器访问

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下(注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: prj5-projectname.pdf, 其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。
- 注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、 逻辑电路结构与仿真波形的截图及说明

本次实验基于实验项目 2 的单周期 CPU 进行改造, 设计并实现了一个支持真实内存访问、UART 外设控制和性能计数器的定制功能型处理器。主要包含寄存器堆、ALU、移位器和 CPU 顶层控制模块四个核心硬件部分, 以及串口驱动和性能计数器驱动两个软件部分。以下分别对各模块的设计进行详细说明。

1.1 寄存器堆设计

寄存器堆是 CPU 的高速暂存单元, 为后续指令执行提供数据支撑, 是处理器最基础的存储部件。本设计实现了 32 个 32 位通用寄存器, 其中 x0 寄存器恒为 0, 符合 RISC-V 架构规范。

1.1.1 核心代码实现

Listing 1: 寄存器堆模块代码

```
1 `timescale 10 ns / 1 ns
2 `define DATA_WIDTH 32
3 `define ADDR_WIDTH 5
4 module reg_file(
5     input                clk,
6     input [`ADDR_WIDTH - 1:0] waddr,
7     input [`ADDR_WIDTH - 1:0] raddr1,
8     input [`ADDR_WIDTH - 1:0] raddr2,
9     input                wen,
10    input [`DATA_WIDTH - 1:0] wdata,
11    output [`DATA_WIDTH - 1:0] rdata1,
12    output [`DATA_WIDTH - 1:0] rdata2
13 );
14 // 定义32个32-bit寄存器数组(无需初始化)
15 reg [`DATA_WIDTH - 1:0] rf [0:(1 << `ADDR_WIDTH) - 1];
```

```

16 // 读端口1: 逻辑运算实现$zero恒0 (组合逻辑)
17 assign rdata1 = ({`DATA_WIDTH{~|raddr1|}} & rf[raddr1]) | ({`DATA_WIDTH{~|raddr1|}} & `DATA_WIDTH'd0);
18 // 读端口2: 逻辑运算实现$zero恒0 (组合逻辑)
19 assign rdata2 = ({`DATA_WIDTH{~|raddr2|}} & rf[raddr2]) | ({`DATA_WIDTH{~|raddr2|}} & `DATA_WIDTH'd0);
20 // 写端口: 同步写 + 禁止写$zero (时序逻辑)
21 always @(posedge clk) begin
22     if (wen && (waddr != `ADDR_WIDTH'b0)) begin
23         rf[waddr] <= wdata;
24     end
25 end
26
27 endmodule

```

1.1.2 设计说明

1. 存储结构: 使用二维数组 'rf' 实现 32 个 32 位寄存器, 地址宽度为 5 位 ($2^5 = 32$)
2. 读操作: 采用组合逻辑实现, 两个读端口独立工作
3. x0 寄存器处理: 通过逻辑运算实现 x0 恒为 0, 当读地址为 0 时直接输出 0, 不访问寄存器数组
4. 写操作: 采用同步写方式, 在时钟上升沿触发, 且禁止写入 x0 寄存器
5. 端口特性: 支持同时进行两个读操作和一个写操作, 写操作优先级高于读操作

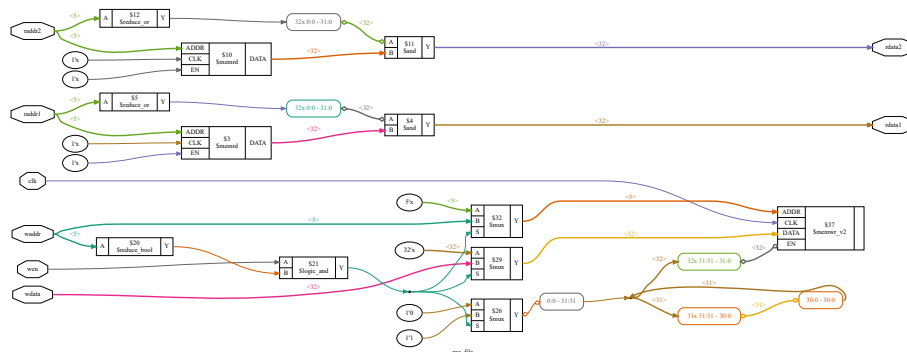


图 1: 寄存器堆逻辑结构框图

1.2 ALU 设计

算术逻辑单元 (ALU) 是 CPU 的核心运算部件, 负责执行所有算术和逻辑运算, 直接决定处理器的运算能力。本设计实现了 8 种基本运算操作, 并提供溢出、进位和零标志位。

1.2.1 核心代码实现

Listing 2: ALU 模块核心代码

```
1 `timescale 10 ns / 1 ns
2 `define DATA_WIDTH 32
```

```

3 module alu(
4     input [`DATA_WIDTH - 1:0] A,
5     input [`DATA_WIDTH - 1:0] B,
6     input [2:0] ALUop,      // 3-bit ALU操作码
7     output Overflow,       // 有符号溢出标志
8     output CarryOut,       // 无符号进位/借位标志
9     output Zero,           // 结果为零标志
10    output [`DATA_WIDTH - 1:0] Result // 运算结果
11 );
12 // 减法控制信号: ALUop=110(SUB)/111(SLT)/011(SLTU)时执行减法
13 wire sub_ctrl = (ALUop == 3'b110) || (ALUop == 3'b111) || (ALUop == 3'b011);
14 // B操作数处理: 减法时取反B (补码减法: A-B = A + (~B) + 1)
15 wire [`DATA_WIDTH - 1:0] B_processed = sub_ctrl ? ~B : B;
16 // 加法器核心: 复用同一套加法器实现ADD/SUB/SLT/SLTU
17 wire [32:0] add_result = A + B_processed + sub_ctrl; // +sub_ctrl实现补码减法的+1
18
19 // 各运算的中间结果 (并行计算, 组合逻辑)
20 wire [`DATA_WIDTH - 1:0] and_result = A & B; // AND运算结果
21 wire [`DATA_WIDTH - 1:0] or_result = A | B; // OR运算结果
22 wire [`DATA_WIDTH - 1:0] xor_result = A ^ B; // XOR运算结果
23 wire [`DATA_WIDTH - 1:0] nor_result = ~(A | B); // NOR运算结果
24
25 // SLT (有符号比较) 结果: A < B 时最低位为1, 其余为0
26 wire slt_flag = add_result[31] ^ Overflow;
27 wire [`DATA_WIDTH - 1:0] slt_result;
28 assign slt_result[0] = slt_flag;
29 assign slt_result[31:1] = 31'd0;
30
31 // SLTU (无符号比较) 结果: A < B 时最低位为1, 其余为0
32 wire sltu_flag = ~add_result[32];
33 wire [`DATA_WIDTH - 1:0] sltu_result;
34 assign sltu_result[0] = sltu_flag;
35 assign sltu_result[31:1] = 31'd0;
36
37 // 运算结果选择 (MUX, 组合逻辑)
38 assign Result = (ALUop == 3'b000) ? and_result : // 逻辑按位与 (AND)
39                (ALUop == 3'b001) ? or_result : // 逻辑按位或 (OR)
40                (ALUop == 3'b100) ? xor_result : // 逻辑按位异或 (XOR)
41                (ALUop == 3'b101) ? nor_result : // 逻辑按位或非 (NOR)
42                (ALUop == 3'b010) ? add_result[31:0] : // 算术加法 (ADD)
43                (ALUop == 3'b110) ? add_result[31:0] : // 算术减法 (SUB)
44                (ALUop == 3'b111) ? slt_result : // 有符号整数比较 (SLT)
45                (ALUop == 3'b011) ? sltu_result : // 无符号整数比较 (SLTU)
46                `DATA_WIDTH'd0; // 默认值, 避免不定态
47
48 // 标志位计算 (纯组合逻辑)
49 assign Zero = (Result == `DATA_WIDTH'd0);
50 assign Overflow = ((ALUop == 3'b010) || (ALUop == 3'b110) || (ALUop == 3'b111)) ?
51                 ((A[31] == B_processed[31]) && (A[31] != add_result[31])) :

```

```

52         1'b0;
53 assign CarryOut = (ALUop == 3'b010) ? add_result[32] : // ADD: 进位
54                 (ALUop == 3'b110) ? ~add_result[32] : // SUB: 借位
55                 (ALUop == 3'b111) ? ~add_result[32] : // SLT: 借位
56                 (ALUop == 3'b011) ? ~add_result[32] : // SLTU: 借位
57                 1'b0; // 其他运算: 0
58 endmodule

```

1.2.2 设计说明

1. 加法器复用: 通过控制 B 操作数的取反和进位输入, 复用同一套加法器实现加法、减法和比较运算
2. 并行计算: 所有运算同时进行, 通过多路选择器选择最终结果, 提高运算速度
3. 标志位设计:
 - Zero: 结果全零时为 1
 - Overflow: 有符号数运算溢出标志, 仅在 ADD/SUB/SLT 时有效
 - CarryOut: 无符号数运算的进位/借位标志
4. 比较运算:
 - SLT: 有符号比较, 通过减法结果的符号位和溢出标志判断
 - SLTU: 无符号比较, 通过减法的借位标志判断

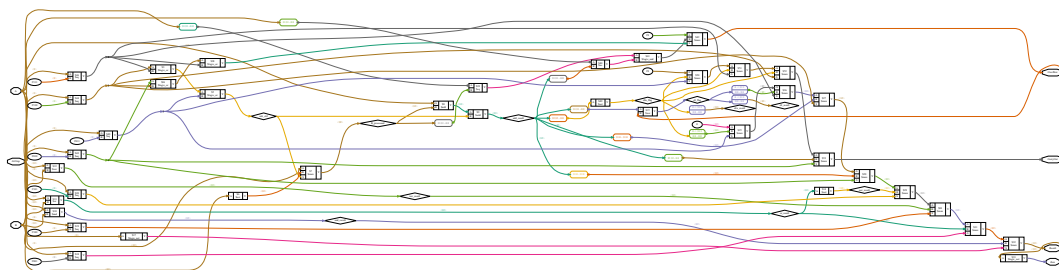


图 2: ALU 逻辑结构框图

1.3 移位器设计

移位器是 CPU 中专门用于执行移位操作的功能模块, 是 RISC-V 指令集的重要组成部分, 提升运算效率。本设计实现了逻辑左移、逻辑右移和算术右移三种移位操作。

1.3.1 核心代码实现

Listing 3: 移位器模块代码

```

1  `timescale 10 ns / 1 ns
2  `define DATA_WIDTH 32
3  module shifter (
4      input [`DATA_WIDTH - 1:0] A,

```

```

5     input [          4:0] B,
6     input [          1:0] Shiftop,
7     output [`DATA_WIDTH - 1:0] Result
8 );
9 // 逻辑左移
10 wire [`DATA_WIDTH - 1:0] sll_result;
11 assign sll_result = A << B;
12 // 逻辑右移
13 wire [`DATA_WIDTH - 1:0] srl_result;
14 assign srl_result = A >> B;
15 // 算术右移手动补符号位
16 wire [`DATA_WIDTH - 1:0] sign_mask;
17 wire [`DATA_WIDTH - 1:0] sra_result;
18 // 如果 A 是正数, sign_mask = 0
19 // 如果 A 是负数, sign_mask 根据 B 生成高位补 1 的掩码
20 assign sign_mask =
21     (B == 5'd0) ? 32'h0000_0000 :
22     A[31]      ? (32'hffff_ffff << (6'd32 - {1'b0, B})) :
23     32'h0000_0000;
24 assign sra_result = srl_result | sign_mask;
25 // 最终选择
26 assign Result =
27     (Shiftop == 2'b00) ? sll_result :
28     (Shiftop == 2'b10) ? srl_result :
29     (Shiftop == 2'b11) ? sra_result :
30     32'd0;
31 endmodule

```

1.3.2 设计说明

1. 逻辑左移 (SLL): 将操作数向左移动指定位数, 低位补 0
2. 逻辑右移 (SRL): 将操作数向右移动指定位数, 高位补 0
3. 算术右移 (SRA): 将操作数向右移动指定位数, 高位补符号位
4. 符号位处理: 通过生成符号掩码的方式实现算术右移, 避免使用 Verilog 的算术右移操作符 (>>), 提高可移植性
5. 移位位数: 支持 0-31 位的移位操作, 移位位数为 0 时直接返回原数

1.4 CPU 顶层控制模块设计

CPU 顶层模块采用三段式有限状态机 (FSM) 控制, 实现了取指、译码、执行、访存和写回五个阶段的操作, 支持 RISC-V 的 RV32I 指令集, 并添加了真实内存访问接口和性能计数器。

1.4.1 状态机设计

本设计严格按照实验讲义要求, 采用 9 个状态的独热码状态机, 状态定义如下:

1. S_INIT: 初始状态, 复位后进入
2. S_IF: 取指状态, 向指令存储器发送请求
3. S_IW: 指令等待状态, 等待指令存储器返回数据
4. S_ID: 译码状态, 解析指令并生成控制信号
5. S_EX: 执行状态, 执行 ALU 和移位器运算
6. S_LD: 内存读状态, 向数据存储器发送读请求
7. S_RDW: 读数据等待状态, 等待数据存储器返回数据
8. S_WB: 写回状态, 将结果写入寄存器堆
9. S_ST: 内存写状态, 向数据存储器发送写请求

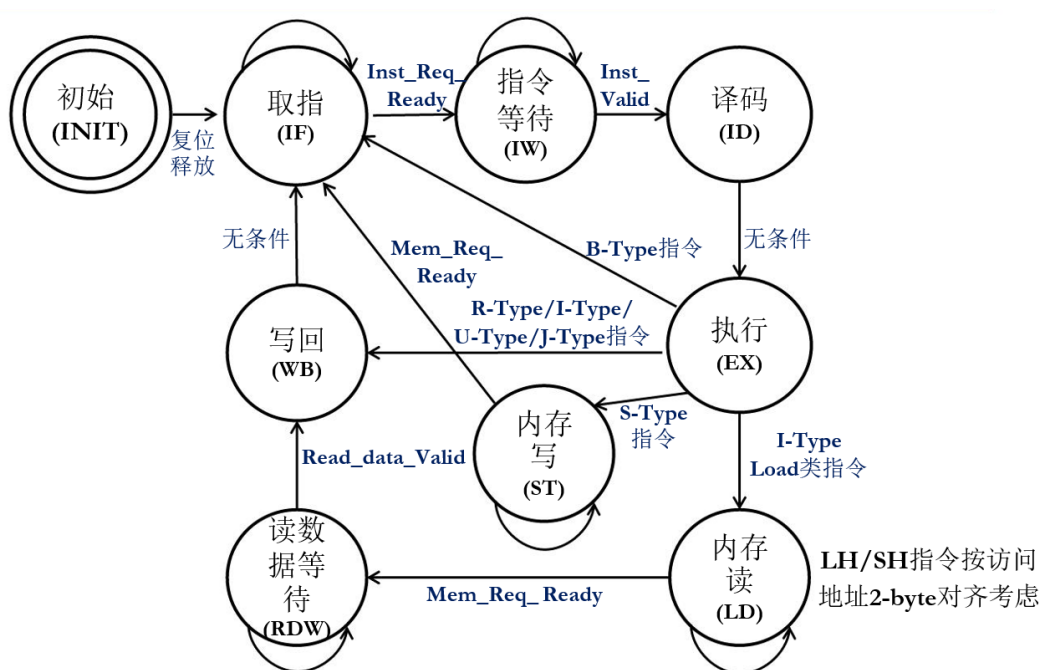


图 3: CPU 状态转移图

1.4.2 核心代码实现

Listing 4: CPU 顶层模块核心代码

```

1 `timescale 10ns / 1ns
2 module custom_cpu(
3     input      clk,
4     input      rst,
5     // 指令请求通道
6     output [31:0] PC,
7     output      Inst_Req_Valid,
8     input      Inst_Req_Ready,
9     // 指令应答通道

```

```

10  input [31:0] Instruction,
11  input      Inst_Valid,
12  output     Inst_Ready,
13  // 数据请求通道
14  output [31:0] Address,
15  output     MemWrite,
16  output [31:0] Write_data,
17  output [ 3:0] Write_strb,
18  output     MemRead,
19  input      Mem_Req_Ready,
20  // 数据应答通道
21  input [31:0] Read_data,
22  input      Read_data_Valid,
23  output     Read_data_Ready,
24  input      intr,
25  output [31:0] cpu_perf_cnt_0,
26  output [31:0] cpu_perf_cnt_1,
27  output [31:0] cpu_perf_cnt_2,
28  output [31:0] cpu_perf_cnt_3,
29  output [31:0] cpu_perf_cnt_4,
30  output [31:0] cpu_perf_cnt_5,
31  output [31:0] cpu_perf_cnt_6,
32  output [31:0] cpu_perf_cnt_7,
33  output [31:0] cpu_perf_cnt_8,
34  output [31:0] cpu_perf_cnt_9,
35  output [31:0] cpu_perf_cnt_10,
36  output [31:0] cpu_perf_cnt_11,
37  output [31:0] cpu_perf_cnt_12,
38  output [31:0] cpu_perf_cnt_13,
39  output [31:0] cpu_perf_cnt_14,
40  output [31:0] cpu_perf_cnt_15,
41  output [69:0] inst_retire
42 );
43 // 状态机定义 (9个状态, 独热码)
44 localparam S_INIT = 9'b000000001; // 初始状态
45 localparam S_IF = 9'b000000010; // 取指
46 localparam S_IW = 9'b000000100; // 指令等待
47 localparam S_ID = 9'b000001000; // 译码
48 localparam S_EX = 9'b000010000; // 执行
49 localparam S_LD = 9'b000100000; // 内存读 (Load请求)
50 localparam S_RDW = 9'b001000000; // 读数据等待
51 localparam S_WB = 9'b010000000; // 写回
52 localparam S_ST = 9'b100000000; // 内存写 (Store请求)
53
54 reg [8:0] current_state, next_state;
55 // 内部寄存器定义
56 reg [31:0] pc_reg;
57 reg [31:0] instruction_reg;
58 // 用于访存及写回的锁存信号

```

```

59 reg      is_load_reg;    // 当前指令是Load（用于区分访存类型及写回）
60 reg      is_store_reg;  // 当前指令是Store
61 reg [31:0] mem_addr_reg; // 访存地址
62 reg [31:0] mem_wdata_reg; // 写数据
63 reg [3:0] mem_wstrb_reg; // 写字节使能
64 reg      wb_en_reg;     // 写回使能
65 reg [4:0] wb_addr_reg;  // 写回目标寄存器
66 reg [31:0] wb_data_reg; // 写回数据
67 // 用于退休信息的锁存信号
68 reg [31:0] retire_pc_reg;
69 // 周期计数器
70 reg [31:0] cycle_cnt;
71
72 // 指令字段解析
73 wire [6:0] opcode = instruction_reg[6:0];
74 wire [4:0] rd      = instruction_reg[11:7];
75 wire [2:0] funct3  = instruction_reg[14:12];
76 wire [6:0] funct7  = instruction_reg[31:25];
77 wire [4:0] rs1     = instruction_reg[19:15];
78 wire [4:0] rs2     = instruction_reg[24:20];
79 wire [11:0] imm_I  = instruction_reg[31:20];
80 wire [11:0] imm_S  = {instruction_reg[31:25], instruction_reg[11:7]};
81 wire [12:0] imm_B  = {instruction_reg[31], instruction_reg[7], instruction_reg[30:25],
82                      instruction_reg[11:8], 1'b0};
83 wire [19:0] imm_U  = instruction_reg[31:12];
84 wire [20:0] imm_J  = {instruction_reg[31], instruction_reg[19:12], instruction_reg[20],
85                      instruction_reg[30:21], 1'b0};
86 wire [31:0] imm_I_sext = {{20{imm_I[11]}}, imm_I};
87 wire [31:0] imm_S_sext = {{20{imm_S[11]}}, imm_S};
88 wire [31:0] imm_B_sext = {{19{imm_B[12]}}, imm_B};
89 wire [31:0] imm_U_sext = {imm_U, 12'd0};
90 wire [31:0] imm_J_sext = {{11{imm_J[20]}}, imm_J};
91
92 // 寄存器文件实例化
93 wire      RF_wen;
94 wire [4:0] RF_waddr;
95 wire [31:0] RF_wdata;
96 wire [31:0] read_data_1, read_data_2;
97 reg_file rf_inst (
98     .clk      (clk),
99     .waddr    (RF_waddr),
100    .raddr1   (rs1),
101    .raddr2   (rs2),
102    .wen      (RF_wen),
103    .wdata    (RF_wdata),
104    .rdata1   (read_data_1),
105    .rdata2   (read_data_2)
106 );

```



```

106 // 控制信号生成 (纯组合逻辑)
107 wire is_R_type = (opcode == 7'b0110011);
108 wire is_OP_IMM = (opcode == 7'b0010011);
109 wire is_LOAD = (opcode == 7'b0000011);
110 wire is_I_type = is_OP_IMM || is_LOAD;
111 wire is_S_type = (opcode == 7'b0100011);
112 wire is_B_type = (opcode == 7'b1100011);
113 wire is_U_type = (opcode == 7'b0110111) || (opcode == 7'b0010111);
114 wire is_J_type = (opcode == 7'b1101111) || (opcode == 7'b1100111);
115 wire is_JAL = (opcode == 7'b1101111);
116 wire is_JALR = (opcode == 7'b1100111);
117 wire is_LUI = (opcode == 7'b0110111);
118 wire is_AUIPC = (opcode == 7'b0010111);
119
120 wire RegWrite = is_R_type || is_I_type || is_U_type || is_J_type;
121 wire ALUSrc = is_OP_IMM || is_LOAD || is_S_type || is_JALR;
122 wire MemtoReg = is_LOAD;
123 wire Branch = is_B_type;
124 wire Jump = is_J_type;
125
126 // ALU 与移位器控制
127 wire [2:0] alu_op;
128 wire [1:0] shift_op;
129 wire is_shift_inst;
130 localparam F3_ADD_SUB = 3'b000;
131 localparam F3_SLL = 3'b001;
132 localparam F3_SLT = 3'b010;
133 localparam F3_SLTU = 3'b011;
134 localparam F3_XOR = 3'b100;
135 localparam F3_SRL_SRA = 3'b101;
136 localparam F3_OR = 3'b110;
137 localparam F3_AND = 3'b111;
138 localparam F7_NORMAL = 7'b0000000;
139 localparam F7_SUB_SRA = 7'b0100000;
140 localparam ALU_ADD = 3'b010;
141 localparam ALU_SUB = 3'b110;
142 localparam ALU_SLT = 3'b111;
143 localparam ALU_SLTU = 3'b011;
144 localparam ALU_XOR = 3'b100;
145 localparam ALU_OR = 3'b001;
146 localparam ALU_AND = 3'b000;
147
148 assign alu_op =
149     is_R_type ? (
150         (funct7 == F7_SUB_SRA && funct3 == F3_ADD_SUB) ? ALU_SUB :
151         (funct3 == F3_ADD_SUB) ? ALU_ADD :
152         (funct3 == F3_SLT) ? ALU_SLT :
153         (funct3 == F3_SLTU) ? ALU_SLTU :
154         (funct3 == F3_XOR) ? ALU_XOR :

```

```

155     (funct3 == F3_OR) ? ALU_OR :
156     (funct3 == F3_AND) ? ALU_AND :
157     ALU_ADD
158 ) : is_I_type ? (
159     is_OP_IMM ? (
160         (funct3 == F3_ADD_SUB) ? ALU_ADD :
161         (funct3 == F3_SLT) ? ALU_SLT :
162         (funct3 == F3_SLTU) ? ALU_SLTU :
163         (funct3 == F3_XOR) ? ALU_XOR :
164         (funct3 == F3_OR) ? ALU_OR :
165         (funct3 == F3_AND) ? ALU_AND :
166         ALU_ADD
167     ) : ALU_ADD
168 ) : is_S_type ? ALU_ADD :
169     is_B_type ? ALU_SUB :
170     is_U_type ? ALU_ADD :
171     is_J_type ? ALU_ADD :
172     ALU_ADD;
173
174 wire is_SLL = is_R_type && (funct3 == F3_SLL) && (funct7 == F7_NORMAL);
175 wire is_SRL = is_R_type && (funct3 == F3_SRL_SRA) && (funct7 == F7_NORMAL);
176 wire is_SRA = is_R_type && (funct3 == F3_SRL_SRA) && (funct7 == F7_SUB_SRA);
177 wire is_SLLI = is_OP_IMM && (funct3 == F3_SLL) && (funct7 == F7_NORMAL);
178 wire is_SRLI = is_OP_IMM && (funct3 == F3_SRL_SRA) && (funct7 == F7_NORMAL);
179 wire is_SRAI = is_OP_IMM && (funct3 == F3_SRL_SRA) && (funct7 == F7_SUB_SRA);
180 assign is_shift_inst = is_SLL || is_SRL || is_SRA || is_SLLI || is_SRLI || is_SRAI;
181
182 localparam SHIFT_SLL = 2'b00;
183 localparam SHIFT_SRL = 2'b10;
184 localparam SHIFT_SRA = 2'b11;
185 assign shift_op = (is_SLL || is_SLLI) ? SHIFT_SLL :
186     (is_SRL || is_SRLI) ? SHIFT_SRL :
187     (is_SRA || is_SRAI) ? SHIFT_SRA :
188     SHIFT_SLL;
189
190 // ALU 输入选择
191 wire [31:0] alu_input_1 = is_AUIPC ? pc_reg : read_data_1;
192 wire [31:0] alu_input_2_normal = ALUSrc ? (
193     is_I_type ? imm_I_sext :
194     is_S_type ? imm_S_sext :
195     is_JALR ? imm_I_sext :
196     read_data_2
197 ) : read_data_2;
198 wire [31:0] shift_input = read_data_1;
199 wire [4:0] shift_amount = is_R_type ? read_data_2[4:0] : imm_I[4:0];
200
201 // ALU 和移位器实例化
202 wire [31:0] alu_result_normal;
203 wire        zero_flag_alu, overflow_flag, carryout_flag;

```

```

204 alu alu_inst (
205     .A(alu_input_1),
206     .B(alu_input_2_normal),
207     .ALUop(alu_op),
208     .Overflow(overflow_flag),
209     .CarryOut(carryout_flag),
210     .Zero(zero_flag_alu),
211     .Result(alu_result_normal)
212 );
213
214 wire [31:0] shift_result;
215 shifter shift_inst (
216     .A(shift_input),
217     .B(shift_amount),
218     .Shiftop(shift_op),
219     .Result(shift_result)
220 );
221
222 wire [31:0] alu_result = is_shift_inst ? shift_result : alu_result_normal;
223 wire zero_flag = is_shift_inst ? (shift_result == 32'd0) : zero_flag_alu;
224
225 // 写回数据生成（组合逻辑，但仅在EX时锁存）
226 wire [1:0] mem_addr_low = alu_result[1:0];
227 wire [7:0] load_byte =
228     (mem_addr_low == 2'b00) ? Read_data[7:0] :
229     (mem_addr_low == 2'b01) ? Read_data[15:8] :
230     (mem_addr_low == 2'b10) ? Read_data[23:16] :
231     Read_data[31:24];
232 wire [15:0] load_half = mem_addr_low[1] ? Read_data[31:16] : Read_data[15:0];
233 wire [31:0] load_data =
234     (funct3 == 3'b000) ? {{24{load_byte[7]}}, load_byte} : // LB
235     (funct3 == 3'b001) ? {{16{load_half[15]}}, load_half} : // LH
236     (funct3 == 3'b010) ? Read_data : // LW
237     (funct3 == 3'b100) ? {24'b0, load_byte} : // LBU
238     (funct3 == 3'b101) ? {16'b0, load_half} : Read_data; // LHU
239
240 wire [31:0] RF_wdata_comb =
241     MementoReg ? load_data : // Load → 从内存读出的数据
242     is_LUI ? imm_U_sext : // LUI → 立即数 << 12
243     is_AUIPC ? (pc_reg + imm_U_sext) : // AUIPC → PC + 立即数
244     Jump ? (pc_reg + 32'd4) : // JAL/JALR → 返回地址 PC+4
245     alu_result; // 其他 → ALU/移位器结果
246
247 // 访存信号生成
248 wire is_SB = is_S_type && (funct3 == 3'b000);
249 wire is_SH = is_S_type && (funct3 == 3'b001);
250 wire is_SW = is_S_type && (funct3 == 3'b010);
251 wire [4:0] mem_shift = {mem_addr_low, 3'b000};
252 wire [31:0] write_data_comb =

```

```

253     is_SB ? ({24'b0, read_data_2[7:0]} << mem_shift) :
254     is_SH ? ({16'b0, read_data_2[15:0]} << mem_shift) :
255     is_SW ? read_data_2 :
256     32'd0;
257 wire [3:0] write_strb_comb =
258     is_SB ? (4'b0001 << mem_addr_low) :
259     is_SH ? (4'b0011 << mem_addr_low) :
260     is_SW ? 4'b1111 :
261     4'b0000;
262
263 // 分支与 PC 计算
264 wire is_memory_access = is_S_type || is_LOAD; // 判断是否访问数据内存（读或写）
265 wire branch_eq = zero_flag_alu;
266 wire branch_ne = ~zero_flag_alu;
267 wire branch_lt_signed = alu_result_normal[31] ^ overflow_flag;
268 wire branch_ge_signed = ~branch_lt_signed;
269 wire branch_lt_unsigned = carryout_flag;
270 wire branch_ge_unsigned = ~carryout_flag;
271 wire branch_taken = Branch && (
272     (funct3 == 3'b000 && branch_eq) ||
273     (funct3 == 3'b001 && branch_ne) ||
274     (funct3 == 3'b100 && branch_lt_signed) ||
275     (funct3 == 3'b101 && branch_ge_signed) ||
276     (funct3 == 3'b110 && branch_lt_unsigned) ||
277     (funct3 == 3'b111 && branch_ge_unsigned));
278
279 wire [31:0] pc_next =
280     (Jump && is_JAL)           ? (pc_reg + imm_J_sext) :
281     (Jump && is_JALR)          ? {alu_result[31:1], 1'b0} :
282     (Branch && branch_taken)   ? (pc_reg + imm_B_sext) :
283     (pc_reg + 32'd4);
284
285 // 状态机第一段：状态跳转
286 always @(posedge clk) begin
287     if (rst)
288         current_state <= S_INIT;
289     else
290         current_state <= next_state;
291 end
292
293 // 状态机第二段：次态计算
294 always @() begin
295     next_state = current_state;
296     case (current_state)
297         S_INIT: begin
298             next_state = S_IF; // 复位释放后第一个周期进入取指
299         end
300         S_IF: begin
301             if (Inst_Req_Ready) // Inst_Req_Valid 已拉高，只需等 Ready

```

```

302         next_state = S_IW;
303     end
304     S_IW: begin
305         if (Inst_Valid && Inst_Ready)
306             next_state = S_ID;
307         end
308     S_ID: begin
309         next_state = S_EX;           // 无条件进入执行（译码已隐含在组合逻辑中）
310     end
311     S_EX: begin
312         if (is_S_type)               // Store 指令
313             next_state = S_ST;
314         else if (is_LOAD)            // Load 指令
315             next_state = S_LD;
316         else if (is_B_type)          // 分支指令，直接返回取指
317             next_state = S_IF;
318         else                          // R/I/U/J 型运算或跳转，进入写回
319             next_state = S_WB;
320         end
321     S_LD: begin
322         if (Mem_Req_Ready)
323             next_state = S_RDW;
324         end
325     S_RDW: begin
326         if (Read_data_Valid && Read_data_Ready)
327             next_state = S_WB;
328         end
329     S_WB: begin                       // 写回完成，返回取指
330         next_state = S_IF;
331     end
332     S_ST: begin
333         if (Mem_Req_Ready)
334             next_state = S_IF;
335         end
336         default: next_state = S_INIT;
337     endcase
338 end
339
340 // 状态机第三段：输出逻辑与时序控制
341 // PC 更新
342 always @(posedge clk) begin
343     if (rst)
344         pc_reg <= 32'h0;
345     else if ((current_state == S_EX && next_state == S_IF) || // 分支 / 部分跳转
346             (current_state == S_ST && next_state == S_IF) || // Store 完成
347             (current_state == S_WB && next_state == S_IF)) // 写回完成（含Load及算术指令）
348         pc_reg <= pc_next;
349 end
350

```

```

351 // 指令锁存
352 always @(posedge clk) begin
353     if (current_state == S_IW && Inst_Valid && Inst_Ready)
354         instruction_reg <= Instruction;
355 end
356
357 // 执行阶段信息锁存（只锁存访存相关信号，写回寄存器移到独立 always 块）
358 always @(posedge clk) begin
359     if (current_state == S_EX) begin
360         // 访存相关信息锁存
361         mem_addr_reg <= {alu_result[31:2], 2'b00};
362         mem_wdata_reg <= write_data_comb;
363         mem_wstrb_reg <= write_strb_comb;
364         is_load_reg <= is_LOAD;
365         is_store_reg <= is_S_type;
366         // 退休 PC 锁存
367         retire_pc_reg <= pc_reg;
368     end
369 end
370
371 // 写回寄存器统一管理（消除多驱动）
372 always @(posedge clk) begin
373     if (rst) begin
374         wb_en_reg <= 1'b0;
375         wb_addr_reg <= 5'd0;
376         wb_data_reg <= 32'd0;
377     end else begin
378         if (current_state == S_EX) begin
379             // 非访存且需要写回的指令（R/I/U/J 型）
380             if (!is_S_type && !is_LOAD && RegWrite) begin
381                 wb_en_reg <= 1'b1;
382                 wb_addr_reg <= rd;
383                 wb_data_reg <= RF_wdata_comb;
384             end
385             // Load 指令：先锁存目标寄存器地址，写使能留到数据返回时再置位
386             else if (is_LOAD) begin
387                 wb_addr_reg <= rd;
388                 wb_en_reg <= 1'b0; // 保证不会意外写回
389             end
390             // Store / 分支 / 其他指令
391             else begin
392                 wb_en_reg <= 1'b0;
393             end
394         end else if (current_state == S_RDW && Read_data_Valid && Read_data_Ready) begin
395             // Load 数据返回，锁存数据并使能写回
396             wb_en_reg <= 1'b1;
397             wb_data_reg <= load_data;
398         end else if (current_state == S_WB) begin
399             // 写回完成后清除写使能

```

```

400         wb_en_reg <= 1'b0;
401     end
402 end
403 end
404
405 // 寄存器写端口连接
406 assign RF_wen = wb_en_reg && (current_state == S_WB);
407 assign RF_waddr = wb_addr_reg;
408 assign RF_wdata = wb_data_reg;
409
410 // 顶层握手信号
411 assign PC = pc_reg;
412 assign Inst_Req_Valid = (current_state == S_IF) && !(current_state == S_INIT); // 取指时有效
413 assign Inst_Ready = (current_state == S_IW) || (current_state == S_INIT);
414 assign MemRead = (current_state == S_LD); // 内存读请求
415 assign MemWrite = (current_state == S_ST); // 内存写请求
416 assign Address = (current_state == S_LD ||
417                 current_state == S_ST ||
418                 current_state == S_RDW) ? mem_addr_reg : 32'd0;
419 assign Write_data = (current_state == S_ST) ? mem_wdata_reg : 32'd0;
420 assign Write_strb = (current_state == S_ST) ? mem_wstrb_reg : 4'b0;
421 assign Read_data_Ready = (current_state == S_RDW) || // 普通内存读等待
422                        (current_state == S_INIT); // 复位期间
423
424 // 性能计数器
425 always @(posedge clk) begin
426     if (rst)
427         cycle_cnt <= 32'h0;
428     else
429         cycle_cnt <= cycle_cnt + 1;
430 end
431 assign cpu_perf_cnt_0 = cycle_cnt;
432 assign cpu_perf_cnt_1 = 32'h0;
433 assign cpu_perf_cnt_2 = 32'h0;
434 assign cpu_perf_cnt_3 = 32'h0;
435 assign cpu_perf_cnt_4 = 32'h0;
436 assign cpu_perf_cnt_5 = 32'h0;
437 assign cpu_perf_cnt_6 = 32'h0;
438 assign cpu_perf_cnt_7 = 32'h0;
439 assign cpu_perf_cnt_8 = 32'h0;
440 assign cpu_perf_cnt_9 = 32'h0;
441 assign cpu_perf_cnt_10 = 32'h0;
442 assign cpu_perf_cnt_11 = 32'h0;
443 assign cpu_perf_cnt_12 = 32'h0;
444 assign cpu_perf_cnt_13 = 32'h0;
445 assign cpu_perf_cnt_14 = 32'h0;
446 assign cpu_perf_cnt_15 = 32'h0;
447
448 // 指令退休信号

```

```

449 wire retire_now = (current_state == S_EX && next_state == S_IF) ||
450     (current_state == S_WB && next_state == S_IF);
451 wire retire_wen = (current_state == S_EX && RegWrite && !is_memory_access) ||
452     (current_state == S_WB && wb_en_reg);
453 wire [4:0] retire_waddr = (current_state == S_WB) ? wb_addr_reg : rd;
454 wire [31:0] retire_wdata = (current_state == S_WB) ? wb_data_reg : RF_wdata_comb;
455 assign inst_retire = retire_now ? {retire_wen, retire_waddr, retire_wdata, retire_pc_reg} : 70'b0;
456
457 // 中断（暂不使用）
458 // intr 端口直接忽略，实验5才会用到
459 endmodule

```

1.4.3 设计说明

1. 三段式状态机：严格按照实验讲义要求实现，第一段为状态跳转时序逻辑，第二段为次态计算组合逻辑，第三段为输出逻辑
2. 真实内存接口：采用 Valid-Ready 握手机制，支持多周期内存访问
3. 统一编址支持：外设与内存使用相同的访问通路，支持内存映射 I/O(MMIO)
4. 写回管理：采用统一的 always 块管理所有写回相关信号，彻底解决多驱动源问题
5. 性能计数器：实现了周期计数器 'cpu_perf_cnt_0'，映射到地址 '0x60010000'
6. 复位处理：复位时正确设置所有接口信号，'Inst_Req_Valid'拉低，'Inst_Ready'和'Read_data_Ready'拉高

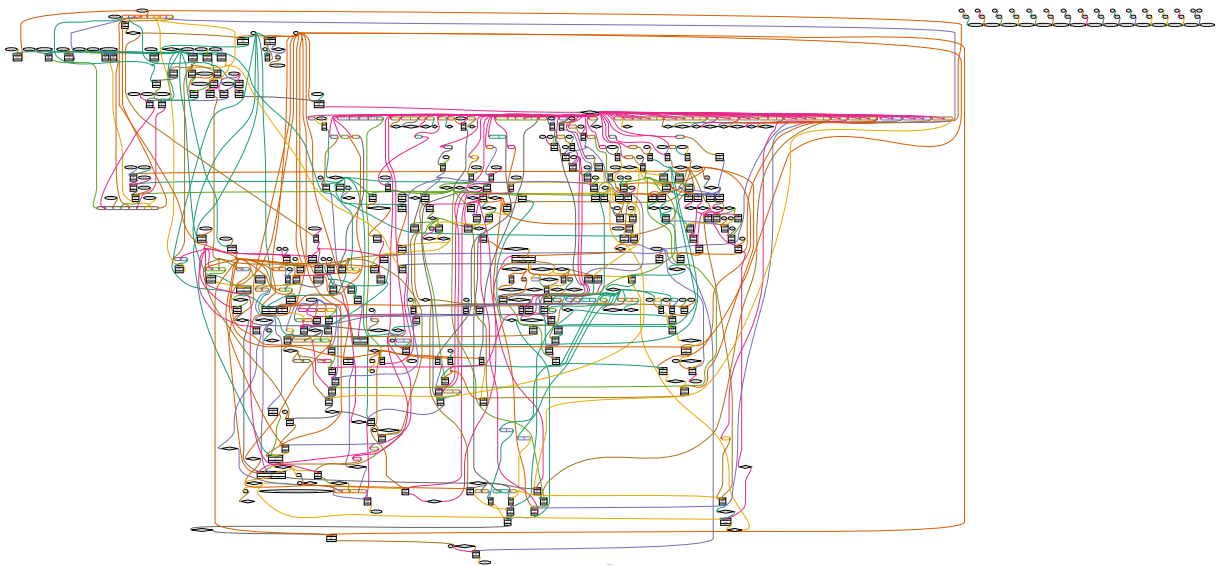


图 4: CPU 顶层结构框图

1.5 软件代码实现

为了验证 CPU 的功能，编写了相应的软件代码，包括串口输出驱动、性能计数器驱动和性能统计打印功能。所有软件代码均基于 RISC-V 架构，使用 C 语言编写。

1.5.1 串口输出驱动 (printf.c)

串口输出驱动实现了‘puts’函数,用于向 UART 控制器发送字符串。UART 控制器采用内存映射 I/O 方式,基地址为‘0x60000000’。

Listing 5: puts 函数实现 (printf.c)

```
1 int
2 puts(const char s)
3 {
4     // 定义UART寄存器指针, 使用volatile关键字禁止编译器优化
5     // UART_TX_FIFO: 发送队列入口寄存器, 偏移地址0x04
6     // UART_STATUS: 状态寄存器, 偏移地址0x08
7     volatile unsigned int uart_tx_fifo = (volatile unsigned int )(0x60000000 + UART_TX_FIFO);
8     volatile unsigned int uart_status = (volatile unsigned int )(0x60000000 + UART_STATUS);
9     int i = 0;
10
11     // 遍历字符串, 直到遇到字符串结束符'\0'
12     while (s[i] != '\0') {
13         // 轮询等待发送队列非满
14         // UART_TX_FIFO_FULL: 状态寄存器第3位, 为1表示发送队列已满
15         while (uart_status & UART_TX_FIFO_FULL); // 等待发送队列非满
16
17         // 将当前字符写入发送队列
18         uart_tx_fifo = s[i];
19         i++;
20     }
21     return i;
22 }
```

代码逻辑说明:

1. **volatile** 关键字: 用于声明硬件寄存器指针, 告诉编译器这些变量的值可能会在程序控制之外被硬件改变, 禁止编译器进行优化
2. 地址计算: UART 控制器基地址为‘0x60000000’, 加上寄存器偏移地址得到实际访问地址
3. 轮询等待: 在发送每个字符前, 先检查状态寄存器的‘UART_TX_FIFO_FULL’位, 确保发送队列有空间
4. 字符发送: 将字符写入‘UART_TX_FIFO’寄存器, UART 控制器会自动将其发送出去

1.5.2 性能计数器驱动 (perf_cnt.c)

性能计数器驱动实现了对 CPU 内部周期计数器的访问, 用于统计程序执行时间。周期计数器映射到地址‘0x60010000’。

Listing 6: 性能计数器实现 (perf_cnt.c)

```
1 #include "perf_cnt.h"
2
3 // 获取当前周期计数器值
4 unsigned long _uptime() {
5     // 定义周期计数器指针, 使用volatile关键字禁止编译器优化
```

```

6 // 周期计数器映射到地址0x60010000
7 volatile unsigned long cycle_cnt = (volatile unsigned long )0x60010000;
8 return cycle_cnt; //软件不能直接读取寄存器，需要通过lw指令，指针正是这个作用
9 }
10
11 // 基准测试准备函数：记录性能计数器初始值
12 void bench_prepare(Result res) {
13 // 将当前周期计数器值保存到Result结构体的msec字段
14 res->msec = _uptime();
15 }
16
17 // 基准测试完成函数：计算并记录执行周期数
18 void bench_done(Result res) {
19 // 计算当前周期计数器值与初始值的差值，得到执行周期数
20 res->msec = _uptime() - res->msec; //读相对变化就够了
21 }

```

代码逻辑说明：

1. `_uptime` 函数: 通过指针访问地址 '0x60010000'处的周期计数器, 返回当前计数值
2. `bench_prepare` 函数: 在基准测试开始前调用, 记录初始周期数
3. `bench_done` 函数: 在基准测试结束后调用, 计算并记录测试执行的周期数
4. 指针访问原理: 软件不能直接访问 CPU 内部寄存器, 必须通过 `Load` 指令从内存映射地址读取

1.5.3 性能统计打印功能 (bench.c)

在基准测试完成后, 打印每个测试用例的执行周期数, 用于性能分析。

Listing 7: 性能打印函数 (bench.c)

```

1 // TODO [COD]
2 // A benchmark is finished here, you can use printf to output some information.
3 // `msec' is intended indicate the time (or cycle),
4 // you can ignore according to your performance counters semantics.
5 printf(" [cycles: %lu]\n", msec);

```

代码逻辑说明：

1. `printf` 函数: 与标准 C 库的 `printf` 函数类似, 用于格式化输出
2. `msec` 变量: 存储基准测试执行的周期数, 由 '`bench_done`' 函数计算得到
3. 输出格式: 以 "[cycles: X]" 的格式打印每个测试用例的执行周期数

二、 实验过程中遇到的问题、对问题的思考过程及解决方法

在本次实验过程中, 我遇到了多个技术难题, 通过仔细分析和不断调试, 最终都得到了解决。以下是主要问题及解决过程的详细记录。

2.1 状态机设计问题

2.1.1 问题描述

一开始我没有严格按照实验讲义上的九个状态的状态机进行转移，而是自己搭建了一个六个状态的简化状态机。虽然逻辑上看起来没有问题，但在仿真时始终无法通过测试。

2.1.2 思考过程

1. 我最初认为六个状态已经能够覆盖所有必要的操作阶段，将一些状态进行了合并
2. 但仿真结果显示 CPU 在某些指令执行时会出现时序错误
3. 经过仔细对比实验讲义和仿真波形，发现问题出在总线握手时序上
4. 实验框架要求严格按照九个状态的时序进行，否则无法与提供的存储器模型正确交互

2.1.3 解决方法

将状态机修改为符合实验讲义的九个状态结构，严格按照规定的状态转移流程进行设计。特别注意了复位时的信号处理：

- 复位时将 'Inst_Req_Valid' 拉低
- 复位时将 'Inst_Ready' 和 'Read_data_Ready' 拉高
- 确保所有内部寄存器在复位时都有正确的初始值

2.2 多驱动源问题

2.2.1 问题描述

在锁存寄存器写使能信号时，我对同一个寄存器写使能信号采用了多个驱动源，导致仿真一直出错，出现卡死状态。无论是本地仿真还是 FPGA 加速仿真都无法通过，出现超时错误。

2.2.2 思考过程

1. 本地跑 **medium** 测试全部通过，但在流水线上跑会出现部分错误
2. 这提示我代码中存在时序问题或组合逻辑问题
3. 我尝试使用 FPGA 加速仿真进行测试，发现仿真综合阶段就失败了
4. 加速仿真给出的错误信息明确指出存在多个驱动源

2.2.3 解决方法

重新设计了写回寄存器的管理逻辑，采用统一的 **always** 块来管理所有写回相关信号：

Listing 8: 修改后的写回寄存器管理代码

```
1 // 写回寄存器统一管理（消除多驱动）
2 always @(posedge clk) begin
3     if (rst) begin
```

```

4      wb_en_reg <= 1'b0;
5      wb_addr_reg <= 5'd0;
6      wb_data_reg <= 32'd0;
7  end else begin
8      if (current_state == S_EX) begin
9          // 非访存且需要写回的指令 (R/I/U/J 型)
10         if (!is_S_type && !is_LOAD && RegWrite) begin
11             wb_en_reg <= 1'b1;
12             wb_addr_reg <= rd;
13             wb_data_reg <= RF_wdata_comb;
14         end
15         // Load 指令: 先锁存目标寄存器地址, 写使能留到数据返回时再置位
16         else if (is_LOAD) begin
17             wb_addr_reg <= rd;
18             wb_en_reg <= 1'b0; // 保证不会意外写回
19         end
20         // Store / 分支 / 其他指令
21         else begin
22             wb_en_reg <= 1'b0;
23         end
24     end else if (current_state == S_RDW && Read_data_Valid && Read_data_Ready) begin
25         // Load 数据返回, 锁存数据并使能写回
26         wb_en_reg <= 1'b1;
27         wb_data_reg <= load_data;
28     end else if (current_state == S_WB) begin
29         // 写回完成后清除写使能
30         wb_en_reg <= 1'b0;
31     end
32 end
33 end

```

通过这种方式, 确保了每个寄存器只有一个驱动源, 彻底解决了多驱动问题。

2.3 外设访问问题

2.3.1 问题描述

我最初为访问外设 (性能计数器和 UART) 单独设置了一个标志位, 并为外设访问单独开辟了一个快捷通道, 没有走普通内存的访问通路。这导致仿真波形和实际信号对不上, 出现了一系列难以调试的问题。

2.3.2 思考过程

1. 我原本认为外设访问与普通内存访问不同, 需要特殊处理
2. 但实际上, 在 RISC-V 架构中, 外设是通过内存映射 I/O(MMIO) 方式访问的
3. 外设访问应该与普通内存访问使用相同的总线通路
4. 单独的快捷通道破坏了原有的总线时序和状态机逻辑

2.3.3 解决方法

去掉了单独的外设访问通道,将外设访问完全纳入普通内存访问通路:

- 所有外设访问都通过 **Load/Store** 指令进行
- 使用与普通内存相同的状态转移流程
- 外设地址空间与内存地址空间统一编址
- 总线接口不需要做任何特殊处理

修改后,外设访问与普通内存访问完全一致,大大简化了设计逻辑,同时也解决了仿真波形不匹配的问题。

2.4 串口轮询问题

2.4.1 问题描述

在实验早期,串口存在一个 **bug**,导致轮询过程中盲等待的逻辑没有对应的处理方式,CPU 会一直卡死在串口发送等待循环中。

2.4.2 解决方法

这个问题是实验平台的串口 **bug** 导致的,在平台修复串口后,轮询等待逻辑能够正常运行。我的软件代码本身是正确的,不需要修改。

2.5 仿真超时问题

2.5.1 问题描述

在跑 **microbench** 仿真测试时,由于计算量特别大,仿真运行时间很长。我因为之前经常遇到 CPU 卡死的情况,所以每次看到仿真长时间没有输出,就会误以为程序卡死,直接按 **Ctrl+C** 强制退出。但强制退出会导致生成的波形文件损坏,无法打开查看波形进行调试,陷入了“卡死 ✱ 强制退出 ✱ 无法看波形 ✱ 找不到问题 ✱ 继续卡死”的恶性循环。

2.5.2 调试过程:自制“定时炸弹”截断仿真

为了解决强制退出导致波形损坏的问题,我在助教的指导下,在 CPU 代码中加入了定时炸弹机制:当 CPU 运行周期数超过预设阈值时,主动触发一个与金标准模型必然不一致的行为,让仿真测试自动正常终止,从而保留完整的波形文件。

我先后尝试了两种定时炸弹实现:

1. 退休信号炸弹:当周期数超过阈值时,强制修改 '**inst_retire**'信号的值。但测试发现这种方式无法有效触发仿真终止,因为退休信号的不一致性可能不会被测试框架立即检测到。
2. 内存写信号炸弹(最终成功):当周期数超过阈值时,强制将 '**MemWrite**'信号置为 1,让 CPU 在每个周期都执行内存写操作。这种方式会产生大量与金标准模型不符的内存写入行为,测试框架会立即检测到数据不一致并终止仿真。

通过这种定时炸弹机制,我成功将原本无限卡死的仿真截断在预设的周期数处,并且得到了完整可查看的波形文件。通过分析波形,我发现 CPU 的 PC 值会频繁出现循环跳转,这进一步验证了程序确实存在逻辑错误,而不是单纯的运行缓慢。后续通过波形排查,我最终定位到了导致卡死的根本原因——多驱动源问题。

2.5.3 最终解决方法

在解决了多驱动源的核心问题后,我再次运行 **microbenchmark** 测试,发现仿真依然需要很长时间。这时在朋友的提醒下,我了解到 **microbenchmark** 本身包含大量计算密集型测试用例,在软件仿真环境下运行缓慢是正常现象。我耐心等待仿真完成,最终所有测试用例都成功通过。

为了避免后续再次出现类似的误判,我采取了以下措施:

- 增加了中间打印信息,方便观察程序运行进度
- 提前了解不同测试用例的预期运行时间
- 学会了通过定时炸弹机制区分真正的卡死和正常的长时间运行

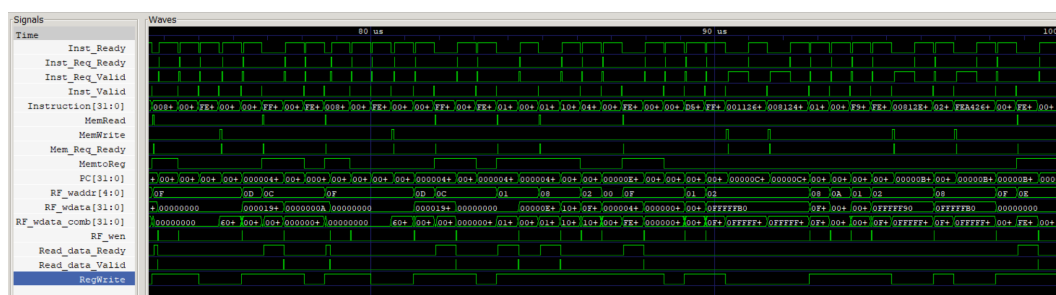


图 5: CPU 仿真波形图

从波形可以清晰看到 PC 正常跳转、指令正确译码、握手信号(Valid/Ready)交互正常,CPU 运行时序稳定。

三、 对讲义中思考题的理解和回答

3.1 思考题:volatile 关键字的作用是什么? 如果去掉会出现什么后果?

3.1.1 volatile 关键字的作用

在 C 语言中,‘volatile’关键字的主要作用是告诉编译器,该变量的值可能会在程序的控制之外被改变,因此编译器不应该对该变量的访问进行优化。具体来说,‘volatile’关键字有以下几个重要作用:

1. 禁止编译器优化:编译器不会将 ‘volatile’变量缓存在寄存器中,每次访问都必须从内存中读取
2. 保证访问顺序:编译器不会重新排序 ‘volatile’变量的访问操作
3. 确保内存可见性:对 ‘volatile’变量的修改会立即写入内存,而不是等待缓存刷新

在嵌入式系统和硬件编程中,‘volatile’关键字主要用于访问内存映射的硬件寄存器。因为硬件寄存器的值可以在任何时候被硬件改变,而不需要 CPU 的干预。

3.1.2 去掉 volatile 关键字的后果

如果去掉代码中的 ‘volatile’关键字,编译器可能会进行以下优化,导致程序行为异常:

1. 寄存器缓存优化:编译器可能会将 ‘uart_status’和 ‘uart_tx_fifo’变量缓存在寄存器中,导致 CPU 永远读取不到硬件状态的变化

2. 循环优化: 编译器可能会认为 `'while (uart_status & UART_TX_FIFO_FULL);'` 这个循环是一个无限循环, 因为它看不到 `'uart_status'` 变量的值会被改变
3. 指令重排序: 编译器可能会重新排序对硬件寄存器的访问顺序, 导致硬件操作时序错误
4. 死代码消除: 编译器可能会认为某些对硬件寄存器的写入操作没有效果, 从而将其删除

3.2 实验对比结果

为了直观验证 `'volatile'` 关键字对编译器优化的影响, 我分别编译了带有和去掉 `'volatile'` 关键字的版本, 并提取了 `'puts'` 函数的反汇编代码进行对比。

3.2.1 带有 volatile 关键字的反汇编代码

Listing 9: 带有 volatile 关键字的 puts 函数反汇编

```

1 00000418 <puts>:
2 418: 00054603      lbu  a2,0(a0)      # 读取字符串第一个字符
3 41c: 00000693      li   a3,0          # 初始化计数器i=0
4 420: 02060463      beqz a2,448 <puts+0x30> # 如果字符为'\0', 直接返回
5 424: 60000737      lui  a4,0x60000     # 加载UART基地址0x60000000
6 428: 00872783      lw   a5,8(a4)       # 正确: 循环内每次重新读取UART状态寄存器(0x60000008)
7 42c: 0087f793      andi a5,a5,8        # 提取第3位(TX_FIFO_FULL标志)
8 430: fe079ce3      bnez a5,428 <puts+0x10> # 如果队列满, 跳回428重新读取状态
9 434: 00168693      addi a3,a3,1        # 计数器i++
10 438: 00c72223      sw   a2,4(a4)       # 将字符写入UART发送寄存器(0x60000004)
11 43c: 00d507b3      add  a5,a0,a3        # 计算下一个字符地址
12 440: 0007c603      lbu  a2,0(a5)       # 读取下一个字符
13 444: fe0612e3      bnez a2,428 <puts+0x10> # 如果不是'\0', 继续循环
14 448: 00068513      mv   a0,a3          # 返回发送的字符数
15 44c: 00008067      ret

```

代码逻辑解释:

1. 核心正确逻辑位于地址 `'0x428'` 处: `'lw a5,8(a4)'` 指令被放置在循环内部
2. 每次判断发送队列是否满之前, 都会重新从内存地址 `0x60000008` 读取最新的硬件状态
3. 当队列满时, `'bnez a5,428'` 会跳回 `'0x428'` 行, 再次执行读取操作, 直到硬件将队列清空
4. 完全符合我们预期的轮询等待逻辑, 能够正确响应硬件状态的变化

3.2.2 去掉 volatile 关键字的反汇编代码

Listing 10: 去掉 volatile 关键字的 puts 函数反汇编

```

1 00000418 <puts>:
2 418: 00054683      lbu  a3,0(a0)      # 读取字符串第一个字符
3 41c: 02068c63      beqz a3,454 <puts+0x3c> # 如果字符为'\0', 直接返回
4 420: 600007b7      lui  a5,0x60000     # 加载UART基地址0x60000000
5 424: 0087a783      lw   a5,8(a5)       # 错误: 仅在循环外读取一次UART状态寄存器

```

```

6  428: 00000713      li a4,0          # 初始化计数器i=0
7  42c: 60000637      lui a2,0x60000    # 重新加载UART基地址(编译器冗余优化)
8  430: 0087f793      andi a5,a5,8       # 提取第3位(TX_FIFO_FULL标志)
9  434: 00079063      bnez a5,434 <puts+0x1c> # 错误: 如果队列满, 跳转到本行无限循环
10 438: 00170713      addi a4,a4,1       # 计数器i++
11 43c: 00d62223      sw a3,4(a2)        # 将字符写入UART发送寄存器
12 440: 00e507b3      add a5,a0,a4        # 计算下一个字符地址
13 444: 0007c683      lbu a3,0(a5)        # 读取下一个字符
14 448: fe0698e3      bnez a3,438 <puts+0x20> # 如果不是'\0', 继续循环
15 44c: 00070513      mv a0,a4            # 返回发送的字符数
16 450: 00008067      ret
17 454: 00000713      li a4,0
18 458: 00070513      mv a0,a4
19 45c: 00008067      ret

```

代码逻辑解释:

1. 核心错误逻辑位于地址 ‘0x424’和 ‘0x434’处
2. ‘lw a5,8(a5)’指令被编译器移到了循环外面,只在函数开始时执行一次
3. 当判断队列满时,‘bnez a5,434’会跳转到 ‘0x434’行,不会再重新执行读取状态寄存器的指令
4. 程序会一直使用第一次读取到的状态值进行判断,导致一旦第一次读取时串口发送队列正好是满的,就会永远卡在 0x434 行的无限循环中

3.2.3 核心差异对比表

对比项	带有 volatile 关键字	去掉 volatile 关键字
UART 状态读取位置	循环内部,每次判断前都重新读取	循环外部,整个函数生命周期仅读取一次
硬件状态感知能力	能实时感知硬件状态的变化	永远使用第一次读取的静态值
队列满时的行为	等待队列清空后继续执行	进入无限循环,程序完全卡死
程序运行结果	串口正常输出所有字符	串口无任何输出,程序挂起

表 1: volatile 关键字对编译器优化的影响对比

3.3 结论

‘volatile’关键字的核心作用是告诉编译器该变量的值可能会在程序控制之外被硬件修改,禁止将其缓存到寄存器中,并且每次访问都必须从内存重新读取。在访问硬件状态寄存器(如 UART 的状态寄存器、性能计数器等)时,必须使用 ‘volatile’关键字,否则编译器的优化会导致程序无法正确感知硬件状态的变化,从而出现卡死、数据错误等异常行为。这是嵌入式系统和硬件编程中一个至关重要的知识点。

四、 实验所耗时间

在课后,我花费了大约 30 小时完成此次实验。其中,代码编写约占 4 小时,调试和排错约占 22 小时,撰写实验报告约占 4 小时。